



CardioGuard AI

Explainable Heart Disease Risk Predictor

Presented By:

Arvind Madaan (2411600)

Janvi Dawra (2411657)

Krishna Sharma (2411681)



The Problem We Solved

The Critical Gap

The healthcare AI ecosystem has made significant progress in **prediction accuracy** but has systematically failed on **explainability and trust**.

Prediction

Tools exist that can predict risk

Explanation ✕

No tools explain *why* the risk exists

- ✕ CardioGuard AI closes this gap — prediction AND explanation, together.



The Problem We Solved



The Critical Gap

The healthcare AI ecosystem has made significant progress in **prediction accuracy** but has systematically failed on **explainability and trust**.

Prediction

Tools exist that can predict risk

Explanation ✗

No tools explain *why* the risk exists

⊗ CardioGuard AI closes this gap — prediction AND explanation, together.

17.9 Million Deaths Annually

Heart disease is the world's leading cause of death according to the WHO — yet most individuals remain unaware of their personal risk until a critical event occurs.



The Problem We Solved



The Critical Gap

The healthcare AI ecosystem has made significant progress in **prediction accuracy** but has systematically failed on **explainability and trust**.

Prediction

Tools exist that can predict risk

Explanation ✗

No tools explain *why* the risk exists

⊗ CardioGuard AI closes this gap — prediction AND explanation, together.

17.9 Million Deaths Annually

Heart disease is the world's leading cause of death according to the WHO — yet most individuals remain unaware of their personal risk until a critical event occurs.

The Black Box Problem

Existing AI diagnostic tools predict outcomes but **cannot explain WHY**. Clinicians and patients cannot trust a model they do not understand — rendering black-box AI clinically inadmissible.



The Problem We Solved



The Critical Gap

The healthcare AI ecosystem has made significant progress in **prediction accuracy** but has systematically failed on **explainability and trust**.

Prediction

Tools exist that can predict risk

Explanation ✗

No tools explain *why* the risk exists

⊗ CardioGuard AI closes this gap — prediction AND explanation, together.

17.9 Million Deaths Annually

Heart disease is the world's leading cause of death according to the WHO — yet most individuals remain unaware of their personal risk until a critical event occurs.

The Black Box Problem

Existing AI diagnostic tools predict outcomes but **cannot explain WHY**. Clinicians and patients cannot trust a model they do not understand — rendering black-box AI clinically inadmissible.

Accessibility Gap

Expensive diagnostic tests — echocardiograms, stress tests, angiography — remain inaccessible in rural and remote areas where cardiac risk is equally prevalent.



CardioGuard AI — What We Built

A full-stack, explainable AI system for early heart disease risk assessment using 13 standard clinical parameters — no expensive diagnostics required.



Risk Prediction

13 standard clinical parameters processed through a trained Random Forest model to generate a precise risk probability score.



Explainable AI (SHAP)

SHAP-powered explanations reveal exactly which features drive each individual prediction — eliminating the black box problem.



Professional Web App

Full login system, patient dashboard, prediction history, analytics, CSV export, and clinical PDF reports.



Probability Assessment

Risk score output with three-tier classification: **Low / Moderate / High** — enabling clear clinical communication.



Personalized Guidance

Clinical recommendations generated dynamically from each patient's specific risk factor profile.



Publicly Deployed

Live application hosted on Vercel + Render + Supabase — accessible to real users in production.

78.7%

Accuracy

Overall classification accuracy on held-out test set

90.9%

Recall

True positive rate — the most critical healthcare metric

87.8%

ROC-AUC

Area under receiver operating characteristic curve

13

Features

Standard clinical parameters used as model inputs



Cleveland Heart Disease Dataset

Dataset Overview

Source

UCI Machine Learning Repository

303 Records · 13 Features

Binary target: Disease / No Disease

Population Statistics



Disease Positive

164 patients



Disease Negative

138 patients

54.4

Avg Age (yrs)

246.5

Avg Chol (mg/dL)

The 13 Clinical Input Features

#	Feature	Description
1	age	Patient's age in years
2	sex	Male or female
3	cp	Type of chest pain experienced
4	trestbps	Resting blood pressure (mmHg)
5	chol	Serum cholesterol (mg/dL)
6	fbs	Fasting blood sugar > 120 mg/dL
7	restecg	Resting electrocardiographic results
8	thalach	Maximum heart rate achieved (bpm)
9	exang	Exercise-induced angina (yes/no)
10	oldpeak	ST depression induced by exercise
11	slope	Slope of the peak exercise ST segment
12	ca	Number of major vessels colored by fluoroscopy
13	thal	Thalassemia blood disorder type

Target: 0 = No Disease | 1 = Disease Present

Team Responsibilities



Clear division of expertise ensured deep specialization across both AI/ML engineering and full-stack web development domains.

Team Responsibilities



Clear division of expertise ensured deep specialization across both AI/ML engineering and full-stack web development domains.

Krishna Sharma & Janvi Dawra — AI & Machine Learning

1 Data Engineering

Data collection, cleaning, duplicate removal, invalid zero handling, and 80/20 stratified train-test split with StandardScaler normalization.

2 Model Development

Exploratory Data Analysis (EDA), training of 5 ML models — Logistic Regression, Decision Tree, Random Forest, KNN, XGBoost — followed by rigorous comparative evaluation.

3 Explainability & Deployment

SHAP TreeExplainer implementation, model persistence using Joblib (model.pkl, scaler.pkl, shap_explainer.pkl), and Streamlit application development.

Team Responsibilities



Clear division of expertise ensured deep specialization across both AI/ML engineering and full-stack web development domains.

Krishna Sharma & Janvi Dawra — AI & Machine Learning

1 Data Engineering

Data collection, cleaning, duplicate removal, invalid zero handling, and 80/20 stratified train-test split with StandardScaler normalization.

2 Model Development

Exploratory Data Analysis (EDA), training of 5 ML models — Logistic Regression, Decision Tree, Random Forest, KNN, XGBoost — followed by rigorous comparative evaluation.

3 Explainability & Deployment

SHAP TreeExplainer implementation, model persistence using Joblib (model.pkl, scaler.pkl, shap_explainer.pkl), and Streamlit application development.

Arvind Madaan — Web Development

1 Frontend Engineering

React 19 + TypeScript application with professional UI/UX design, login and registration system, prediction studio with 13 clinical inputs, and clinical PDF report generator.

2 Backend & Database

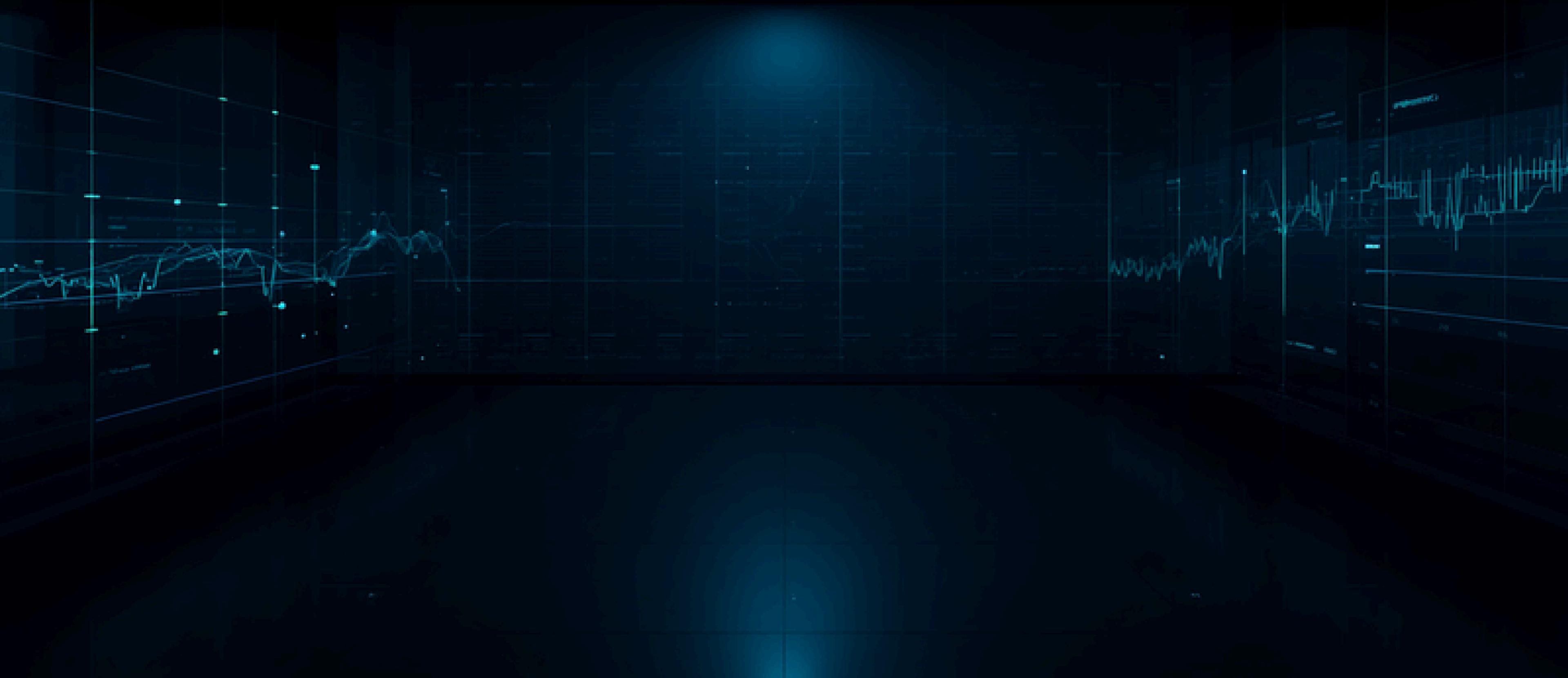
Node.js + Express REST API with persistent Python ML worker process. Supabase + PostgreSQL integration for user profiles, role-based access, prediction history, and audit logs.

3 Deployment & Integration

Vercel (frontend) + Render (backend) + Supabase (database) deployment pipeline with full REST API integration between React frontend and Python ML backend.

Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.



Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.

1

1. Data Loading

Pandas reads **heart.csv** from UCI Cleveland dataset

Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.

1

1. Data Loading

Pandas reads **heart.csv** from UCI Cleveland dataset

2

2. EDA

Statistical analysis and visualization — Matplotlib, Seaborn, Plotly

Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.

1

1. Data Loading

Pandas reads **heart.csv** from UCI Cleveland dataset

2

2. EDA

Statistical analysis and visualization — Matplotlib, Seaborn, Plotly

3

3. Preprocessing

Remove duplicates · Handle invalid zeros · 80/20 stratified split · StandardScaler normalization

Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.

1

1. Data Loading

Pandas reads **heart.csv** from UCI Cleveland dataset

2

2. EDA

Statistical analysis and visualization — Matplotlib, Seaborn, Plotly

3

3. Preprocessing

Remove duplicates · Handle invalid zeros · 80/20 stratified split · StandardScaler normalization

4

4. Model Training

Logistic Regression · Decision Tree · **Random Forest** ✓ · KNN
· XGBoost

Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.

1

1. Data Loading

Pandas reads **heart.csv** from UCI Cleveland dataset

2

2. EDA

Statistical analysis and visualization — Matplotlib, Seaborn, Plotly

3

3. Preprocessing

Remove duplicates · Handle invalid zeros · 80/20 stratified split · StandardScaler normalization

4

4. Model Training

Logistic Regression · Decision Tree · **Random Forest** ✓ · KNN
· XGBoost

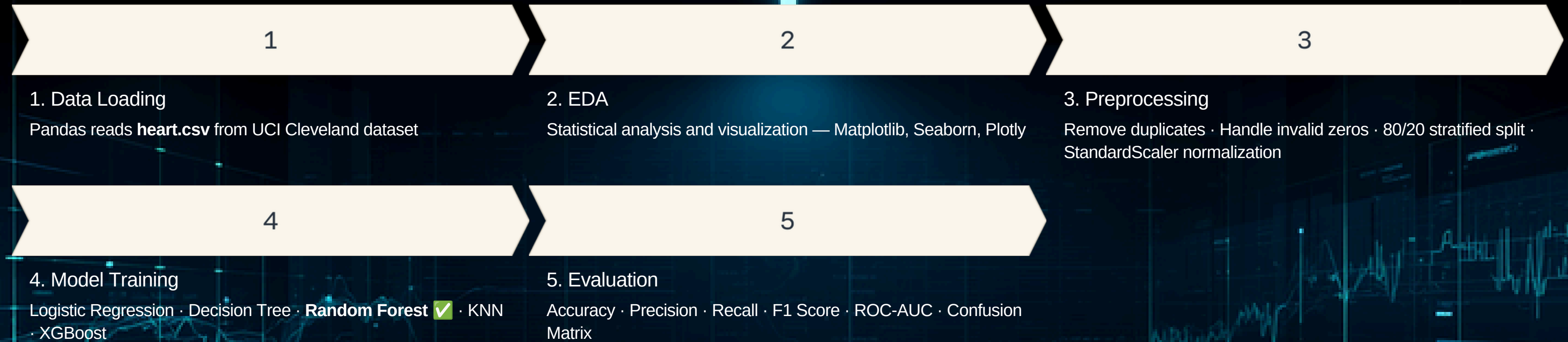
5

5. Evaluation

Accuracy · Precision · Recall · F1 Score · ROC-AUC · Confusion Matrix

Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.

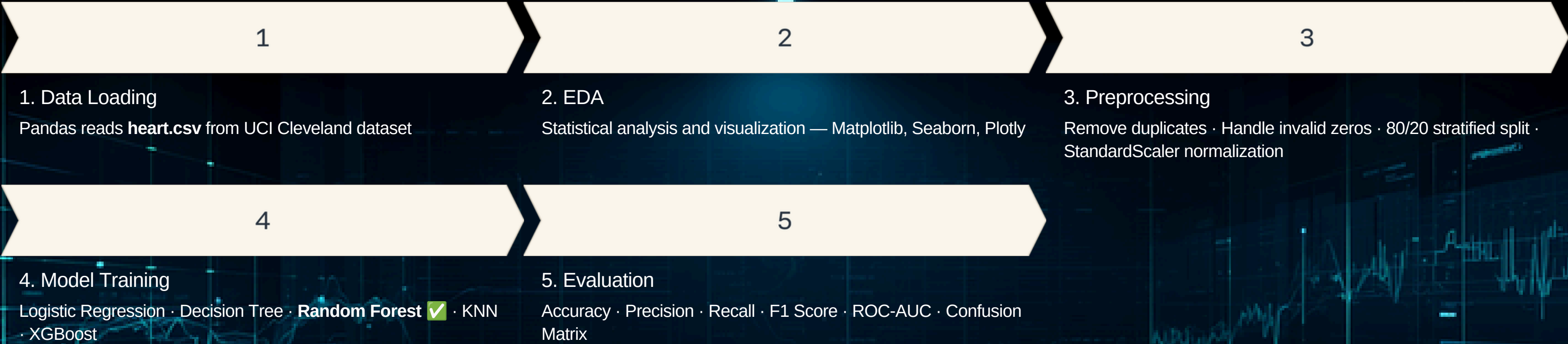


Step 6: SHAP Explainability

TreeExplainer is fitted on the trained Random Forest model and serialized alongside it. For every new prediction, the explainer computes individual Shapley values — showing exactly which features pushed the model toward or away from a disease prediction. This is the core transparency mechanism that differentiates CardioGuard AI from black-box alternatives.

Our Complete Machine Learning Pipeline

A seven-stage pipeline from raw CSV data to production-ready deployed model, implementing ML engineering best practices throughout.



Step 6: SHAP Explainability

TreeExplainer is fitted on the trained Random Forest model and serialized alongside it. For every new prediction, the explainer computes individual Shapley values — showing exactly which features pushed the model toward or away from a disease prediction. This is the core transparency mechanism that differentiates CardioGuard AI from black-box alternatives.

Step 7: Model Deployment

Three artifacts are saved using Joblib for production use:

model.pkl

Trained Random Forest classifier

scaler.pkl

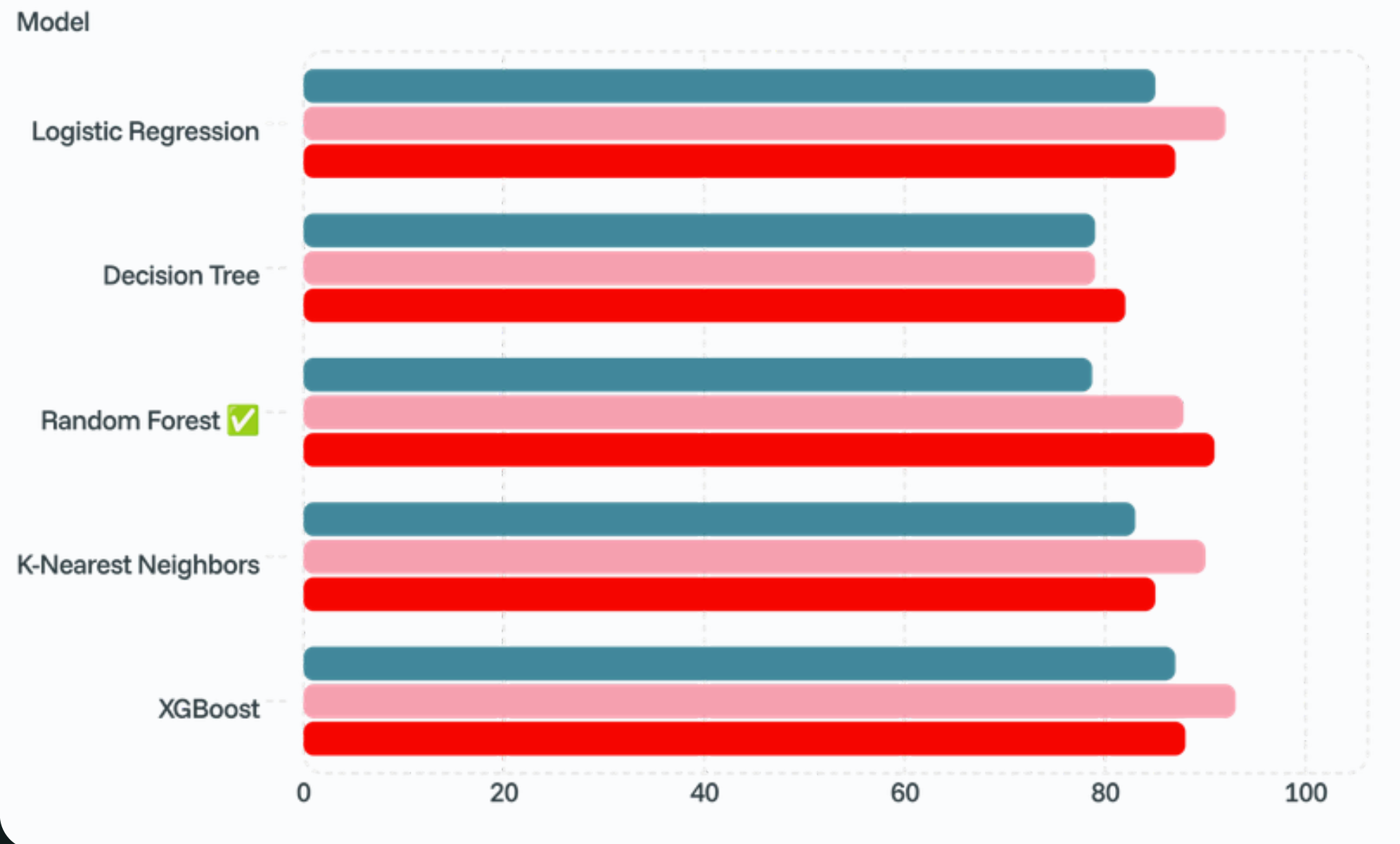
Fitted StandardScaler for input normalization

shap_explainer.pkl

TreeExplainer for live SHAP computation

Model Performance Comparison

Comparison Across 5 Models



Why Random Forest Won

90.9% Recall — Highest of All Models

Minimizes false negatives — missed disease cases that could prove dangerous to patients.

0.878 ROC-AUC — Excellent Discrimination

Strong ability to distinguish between disease-positive and disease-negative patients.

Overfitting Resistance

Ensemble of decision trees generalizes well on small datasets like the 303-record Cleveland set.

Built-in Interpretability

Native feature importance compatible with SHAP TreeExplainer for transparent predictions.

The Healthcare Recall Imperative

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

False Negative

Missed disease → **Potentially life-threatening**

False Positive

Extra testing → **Usually less harmful**

SHAP Explainability

What is SHAP?

SHAP = SHapley Additive exPlanations

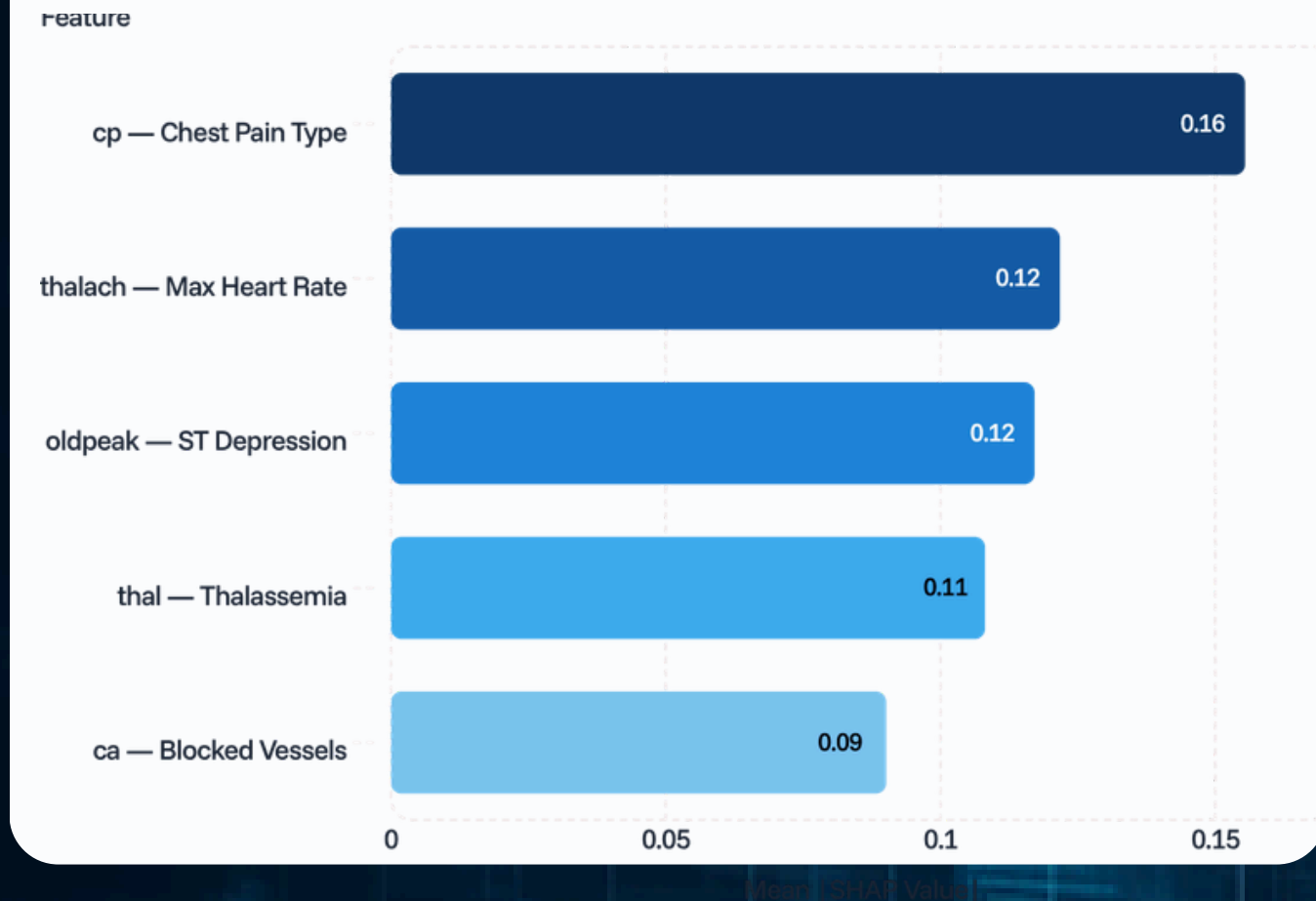
Grounded in cooperative game theory (Shapley values), SHAP is a mathematically principled framework for attributing each feature's contribution to an individual prediction — not just a global average, but a per-patient, per-prediction explanation.

Why SHAP Matters

- Explains every individual prediction uniquely
- Shows which features **increase** disease risk (positive SHAP)
- Shows which features **decrease** disease risk (negative SHAP)
- Builds clinician and patient trust through full transparency

📄 **Positive SHAP:** Feature pushes toward disease risk | **Negative SHAP:** Feature pushes away from disease risk

Top 5 Features by SHAP Importance



📄 *Example: High chest pain type (cp) is the single most influential predictor — a SHAP value of 0.1554 means it shifts the model's output by ~15.5 percentage points toward disease risk on average.*

Full Stack Web Application

CardioGuard AI is deployed as a production-grade, multi-tier web application — not a prototype or notebook demo.



Full Stack Web Application

CardioGuard AI is deployed as a production-grade, multi-tier web application — not a prototype or notebook demo.

Frontend

React 19 + TypeScript

- Landing page with live metrics
- Login and registration system
- Patient dashboard with analytics
- Prediction studio — 13 clinical inputs
- Prediction history browser
- CSV export functionality
- Data insights and visualizations
- Clinical PDF report generator

Full Stack Web Application

CardioGuard AI is deployed as a production-grade, multi-tier web application — not a prototype or notebook demo.

Frontend

React 19 + TypeScript

- Landing page with live metrics
- Login and registration system
- Patient dashboard with analytics
- Prediction studio — 13 clinical inputs
- Prediction history browser
- CSV export functionality
- Data insights and visualizations
- Clinical PDF report generator

Backend

Node.js + Express

- REST API endpoints for all operations
- Persistent Python ML worker process via stdio pipe
- Dynamic benchmark service
- Clinical recommendations engine

Full Stack Web Application

CardioGuard AI is deployed as a production-grade, multi-tier web application — not a prototype or notebook demo.

Frontend

React 19 + TypeScript

- Landing page with live metrics
- Login and registration system
- Patient dashboard with analytics
- Prediction studio — 13 clinical inputs
- Prediction history browser
- CSV export functionality
- Data insights and visualizations
- Clinical PDF report generator

Backend

Node.js + Express

- REST API endpoints for all operations
- Persistent Python ML worker process via stdio pipe
- Dynamic benchmark service
- Clinical recommendations engine

Database

Supabase + PostgreSQL

- User profiles with role-based access
- Prediction history and audit logs
- Real-time notifications

Full Stack Web Application

CardioGuard AI is deployed as a production-grade, multi-tier web application — not a prototype or notebook demo.

Frontend

React 19 + TypeScript

- Landing page with live metrics
- Login and registration system
- Patient dashboard with analytics
- Prediction studio — 13 clinical inputs
- Prediction history browser
- CSV export functionality
- Data insights and visualizations
- Clinical PDF report generator

Backend

Node.js + Express

- REST API endpoints for all operations
- Persistent Python ML worker process via stdio pipe
- Dynamic benchmark service
- Clinical recommendations engine

Database

Supabase + PostgreSQL

- User profiles with role-based access
- Prediction history and audit logs
- Real-time notifications

Deployment

Three-Platform Production Stack

- **Vercel** — Frontend hosting with global CDN
- **Render** — Backend Node.js + Python ML worker
- **Supabase** — Managed PostgreSQL database

Complete System Architecture

End-to-End Data Flow — Training Phase → Prediction Phase → Response

Training Phase

01

data/heart.csv

303-record Cleveland dataset loaded via Pandas

02

Python ML Pipeline

Training, evaluation, and SHAP explainer fitting

03

models/*.pkl

model.pkl · scaler.pkl · shap_explainer.pkl saved via Joblib

Technology Stack

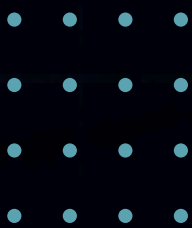
ML: Python · scikit-learn · SHAP

Frontend: React 19 · TypeScript

Backend: Node.js · Express

DB: PostgreSQL · Supabase

Deploy: Vercel · Render



Complete System Architecture

End-to-End Data Flow — Training Phase → Prediction Phase → Response

Training Phase

01

data/heart.csv

303-record Cleveland dataset loaded via Pandas

02

Python ML Pipeline

Training, evaluation, and SHAP explainer fitting

03

models/*.pkl

model.pkl · scaler.pkl · shap_explainer.pkl saved via Joblib

Technology Stack

ML: Python · scikit-learn · SHAP

Frontend: React 19 · TypeScript

Backend: Node.js · Express

DB: PostgreSQL · Supabase

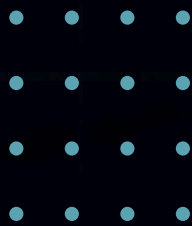
Deploy: Vercel · Render

Prediction Phase — Live Request Flow



Patient Input

13 clinical parameters entered in Prediction Studio (React frontend on Vercel)



Complete System Architecture

End-to-End Data Flow — Training Phase → Prediction Phase → Response

Training Phase

01

data/heart.csv

303-record Cleveland dataset loaded via Pandas

02

Python ML Pipeline

Training, evaluation, and SHAP explainer fitting

03

models/*.pkl

model.pkl · scaler.pkl · shap_explainer.pkl saved via Joblib

Technology Stack

ML: Python · scikit-learn · SHAP

Frontend: React 19 · TypeScript

Backend: Node.js · Express

DB: PostgreSQL · Supabase

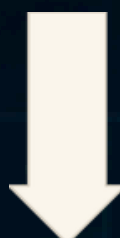
Deploy: Vercel · Render

Prediction Phase — Live Request Flow



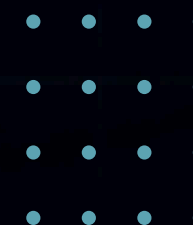
Patient Input

13 clinical parameters entered in Prediction Studio (React frontend on Vercel)



POST Request → Node.js Express Backend (Render)

REST API receives structured JSON payload; validates and pipes to Python worker



Complete System Architecture

End-to-End Data Flow — Training Phase → Prediction Phase → Response

Training Phase

01

data/heart.csv

303-record Cleveland dataset loaded via Pandas

02

Python ML Pipeline

Training, evaluation, and SHAP explainer fitting

03

models/*.pkl

model.pkl · scaler.pkl · shap_explainer.pkl saved via Joblib

Technology Stack

ML: Python · scikit-learn · SHAP

Frontend: React 19 · TypeScript

Backend: Node.js · Express

DB: PostgreSQL · Supabase

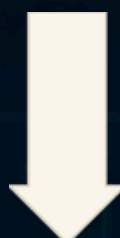
Deploy: Vercel · Render

Prediction Phase — Live Request Flow



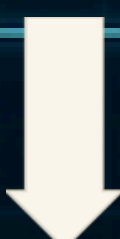
Patient Input

13 clinical parameters entered in Prediction Studio (React frontend on Vercel)



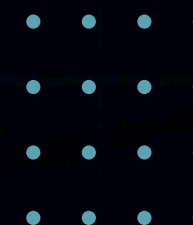
POST Request → Node.js Express Backend (Render)

REST API receives structured JSON payload; validates and pipes to Python worker



Python predict_worker.py → Random Forest Model

Loads model.pkl + scaler.pkl; normalizes input; generates prediction probability



Complete System Architecture

End-to-End Data Flow — Training Phase → Prediction Phase → Response

Training Phase

01

data/heart.csv

303-record Cleveland dataset loaded via Pandas

02

Python ML Pipeline

Training, evaluation, and SHAP explainer fitting

03

models/*.pkl

model.pkl · scaler.pkl · shap_explainer.pkl saved via Joblib

Technology Stack

ML: Python · scikit-learn · SHAP

Frontend: React 19 · TypeScript

Backend: Node.js · Express

DB: PostgreSQL · Supabase

Deploy: Vercel · Render

Prediction Phase — Live Request Flow

↓

Patient Input

13 clinical parameters entered in Prediction Studio (React frontend on Vercel)

↓

POST Request → Node.js Express Backend (Render)

REST API receives structured JSON payload; validates and pipes to Python worker

↓

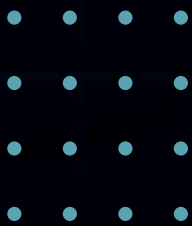
Python predict_worker.py → Random Forest Model

Loads model.pkl + scaler.pkl; normalizes input; generates prediction probability

↓

SHAP Explanation + Clinical Recommendations Engine

shap_explainer.pkl computes per-feature Shapley values; recommendations engine generates personalized clinical guidance



Complete System Architecture

End-to-End Data Flow — Training Phase → Prediction Phase → Response

Training Phase

01

data/heart.csv

303-record Cleveland dataset loaded via Pandas

02

Python ML Pipeline

Training, evaluation, and SHAP explainer fitting

03

models/*.pkl

model.pkl · scaler.pkl · shap_explainer.pkl saved via Joblib

Technology Stack

ML: Python · scikit-learn · SHAP

Frontend: React 19 · TypeScript

Backend: Node.js · Express

DB: PostgreSQL · Supabase

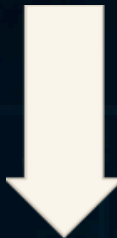
Deploy: Vercel · Render

Prediction Phase — Live Request Flow



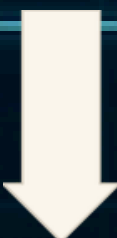
Patient Input

13 clinical parameters entered in Prediction Studio (React frontend on Vercel)



POST Request → Node.js Express Backend (Render)

REST API receives structured JSON payload; validates and pipes to Python worker



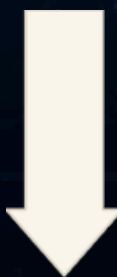
Python predict_worker.py → Random Forest Model

Loads model.pkl + scaler.pkl; normalizes input; generates prediction probability



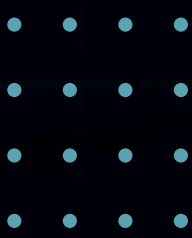
SHAP Explanation + Clinical Recommendations Engine

shap_explainer.pkl computes per-feature Shapley values; recommendations engine generates personalized clinical guidance



Supabase PostgreSQL → REST API Response → Patient Dashboard

Prediction, SHAP values, and recommendations stored in audit log; full result returned and rendered in patient dashboard



Live Application & Demo

Access CardioGuard AI

 Web Application
cardioguard20.vercel.app

 Streamlit Version
cardioguard-ai.streamlit.app

 GitHub Repository
github.com/krisha1904/CardioGuard-AI

78.7%

Accuracy

90.9%

Recall

87.8%

ROC-AUC

Live Demo Walkthrough

01

Landing Page

Show live metrics display, system status indicators, and application overview

02

Authentication

Login as patient with credentials — demonstrates full auth system and role-based access

03

Prediction Studio

Enter all 13 clinical parameters — shows the complete clinical input interface

04

Risk Prediction Result

Display risk classification (Low / Moderate / High) with precise probability score

05

SHAP Explanation

Show per-feature Shapley value visualization — the transparency differentiator

06

Patient History & Analytics

Browse prediction history, trends over time, and full data analytics dashboard

The background of the slide is a dark blue medical monitor interface. It features several ECG (heart rate) waveforms in a light blue color. There are also various data panels and controls visible, including a top-left panel with a grid of dots, a top-right panel with a play button, and a bottom-right panel with a list of items. The overall aesthetic is technical and medical.

Thank You

Any Question?